Physics-Informed Neural Networks for Mechanics: Reproduction, Diagnosis and Improvements

Daruru Bhuvan Sai (PES1UG24AM356) and Vismay H S (PES1UG24AM325)

UE24CS352A Machine Learning, mini-project report. PES University, October 2026.

Abstract

Physics-informed neural networks (PINNs) train a neural network to satisfy a differential equation by adding the equation's residual to the loss. Zhang, Chen and Yang [1] used them on three problems in mechanics: consolidation of a soil layer, steady heat conduction in a plate, and recovering the damping and stiffness of an oscillator from measurements. They reported one training run for each problem. We rebuilt their setup and repeated every experiment over 5 to 10 random seeds, measuring the error against exact solutions. The reported accuracy holds for some seeds and not for others. In each problem the remaining error sits in a small region that a smooth network cannot represent well: a slowly decaying tail, a jump in the initial condition, or the corners of a square.

We removed each of these errors by building the known local behaviour into the network itself. A two-stage warm start finds the oscillator parameters in 10 of 10 seeds instead of 3 of 5, and runs 30 times faster. An output enriched with an exact erf term lowers the consolidation error about 200-fold. An exact boundary condition followed by analytical corner terms lowers the worst-case heat error about 140-fold. Finally, on noisy and sparse oscillator data, a classical least-squares fit of the ODE was more accurate than every PINN variant we tested.

1. Introduction

Many problems in engineering mechanics are governed by equations that are known exactly, while the measurements available for a particular soil sample or structure are few and noisy. PINNs [2] try to use both. The network is trained to match the measurements that exist and, at the same time, to satisfy the governing equation at a set of collocation points. Derivatives come from automatic differentiation, so the same code handles forward problems (predict the field) and inverse problems (recover unknown coefficients) without a mesh.

Our base paper [1], a Stanford CS229 project, applies PINNs to three classical problems and concludes that the predictions match the references well. Each conclusion rests on a single training run. We wanted to know three things:

1. Do the reported results hold across random seeds?
2. Where does the remaining error sit, and what causes it?
3. Can a change to the method remove that cause, and what does the change cost?

The paper also proposes two improvements that it does not try: exact boundary conditions and adaptive placement of training points. We tested both. We also added a study of the oscillator problem with noisy and sparse data, which the paper does not cover.

Section 2 introduces PINNs and the three problems. Section 3 describes our implementation and how we evaluate it. Section 4 presents the results problem by problem. Section 5 compares our work with the base paper and discusses its limits, and Section 6 concludes. Appendix A explains how to reproduce every number in the report.

2. Background

2.1 Physics-informed neural networks

A PINN is an ordinary fully connected network N. It takes the coordinates (for example x and t) as input and returns the value of the solution. Because N is differentiable, automatic differentiation gives every derivative that appears in the equation, so the residual r of the equation can be computed at any point. Training minimises a weighted sum of mean-squared terms:

$$L = w_{PDE} \cdot \mathrm{mean}(r^2) + w_{BC} \cdot \mathrm{mean}((N - g)^2) + w_{IC} \cdot \mathrm{mean}((N - p_0)^2) + w_{data} \cdot \mathrm{mean}((N - u_{obs})^2) \quad (1)$$

The residual r is computed at randomly placed collocation points, and g, p_0 and u_obs are the boundary values, initial values and measurements. A problem uses only the terms that apply to it. In an inverse problem, the unknown coefficients of the equation are trained together with the network weights.

In equation (1) the boundary and initial conditions are only penalties, so they hold approximately. An older alternative, due to Lagaris et al. [3], writes the output so that it satisfies them exactly. For example, the network can be multiplied by a function that is zero on the boundary. Both of our main improvements build on this idea.

2.2 The three problems

Table 1 lists the problems as the paper poses them. All three have exact solutions, which we use as references throughout.

Table 1. The three problems studied in the base paper.

Paper section	Problem	Task	Equation and conditions
4.1	One-dimensional (Terzaghi [8]) consolidation of a soil layer	Forward: predict pore pressure p(x, t)	p_t = p_xx on 0 < x < 1, 0 < t ⩽ 0.5; p = 0 at the drained top, p_x = 0 at the sealed base, p = 1 at t = 0
4.2	Steady heat conduction with a uniform source	Forward: predict temperature T(x, y)	T_xx + T_yy + 1 = 0 on the square $(-1, 1)^2$; T = 0 on the edges
4.3	Damped mass-spring oscillator	Inverse: find damping c and stiffness k	u'' + c u' + k u = 0, u(0) = u'(0) = 1, t in [0, 15]; 50 measurements of u; true c = 0.4, k = 4; initial guess c = 0.2, k = 2

3. Method

3.1 Implementation

We wrote one PINN engine in PyTorch and used it for every experiment. Network sizes, point counts, loss weights and learning rates follow the paper (Table 2). The networks use tanh activations, because the equations need second derivatives and ReLU does not have useful ones. Inputs are scaled to [−1, 1], and weights use Xavier initialisation [7].

Training follows the usual PINN recipe [2]: Adam [5] first, to move away from the random start, and then L-BFGS [6] to converge. If L-BFGS diverges, the engine restores the last good weights. Everything runs in float64. L-BFGS needs the extra precision near convergence, and on our RTX 4050 a float64 step cost about the same as a float32 one (12.2 ms and 12.6 ms). The engine logs each loss term separately, so when a run fails we can tell whether the equation, a boundary, the initial condition or the data term is responsible.

Table 2. Network and training settings, taken from the base paper.

Problem	Network (width x layers)	Training points	Optimiser (paper)	Learning rate
Oscillator	32 x 3	50 measurements, 100 collocation	Adam 100k	5e-4
Consolidation	50 x 5	500 collocation, 250 boundary, 125 initial	Adam 50k + L-BFGS	1e-3
Heat	50 x 4	1,500 collocation, 500 boundary	Adam 50k + L-BFGS	5e-4

3.2 Evaluation

Our evaluation differs from the paper's in three ways.

First, every configuration runs over 5 random seeds, and over 10 for the oscillator fixes. We report the median and the worst run. A single run can mislead: one shortcut we tried succeeded on 5 of 5 seeds, but only on 8 of 10 when we doubled the count.

Second, every error is measured against an exact solution. For the oscillator this is the closed form. For consolidation it is the method-of-images (erfc) solution [9], which we checked against the paper's Fourier series. For heat it is the classical series solution, the same function that describes torsion of a square bar [10], which we checked against our own finite-difference solver. The paper states that the heat problem has no analytical solution, but it does.

Third, we look at where the error sits, not only at its size. Besides the relative L2 error and the maximum error, we measure the error near corners and edges and at early times. We also compute the equation residual on 20,000 fresh points and compare it with the residual at the training points. A large gap means the training loss is hiding an error that the network makes between its training points.

Each improvement in Section 4 came out of this evaluation. It showed where the error was, and the fix then changed either the network's output (consolidation and heat) or its starting point (oscillator).

4. Results

4.1 Damped oscillator: finding c and k

With the paper's settings, only 3 of 5 seeds find both parameters within 1%, and those 3 need a median of 71,000 Adam iterations. The paper's single run (c = 0.40125, k = 3.9933) was one of the good ones. The stiffness k is always found. The damping c is the problem.

To see why, we tracked the values of c and k that the network's current curve implies, by a least-squares fit of the equation to the network's own derivatives. Throughout training, the trainable c and k stayed equal to these implied values. So c is wrong whenever the curve is wrong, and the curve is wrong for a long time. The network learns the signal from left to right, and until it reaches the end, the tail of the curve is flat. A flat tail looks like heavy damping, with c near 1. The small late oscillations that carry the information about c hold only 1% of the signal's energy, so the data term barely pulls on them. The physics term does not help either. The flat solution u = 0 satisfies the equation exactly, and early in training c and k still sit at their wrong starting values (0.2 and 2), so the physics term pulls the curve toward the wrong solution.

The fix is to stop the physics term from acting before the curve is right. Our two-stage warm start works as follows:

1. Fit the network to the measurements and the initial condition only. With no physics term, nothing rewards a flat tail.
2. Read c and k off the fitted curve by least squares on its derivatives.
3. Train the full PINN with c and k frozen at those values.
4. Release c and k and train everything jointly.

Table 3 compares the paper's setup with a plain Adam-then-L-BFGS run, our method and two ablations that each remove one step.

Table 3. Oscillator results over 10 seeds (5 for the paper's setup). Errors are relative.

Method	Seeds within 1%	c error, median (worst)	k error, median (worst)	Time per run
Paper (Adam 100k)	3/5	9.1e-3 (4.0e-2)	1.0e-3 (4.1e-3)	699 s
Adam 20k + L-BFGS	8/10	7.3e-3 (1.1e-1)	5.5e-4 (4.1e-3)	59 s
Two-stage without step 2	7/10	7.3e-4 (5.2e-2)	4.6e-4 (1.0e-2)	22 s
Two-stage without step 3	10/10	1.9e-3 (7.2e-3)	1.8e-4 (1.0e-3)	23 s
Two-stage (full)	10/10	1.4e-3 (5.6e-3)	4.4e-4 (1.1e-3)	23 s

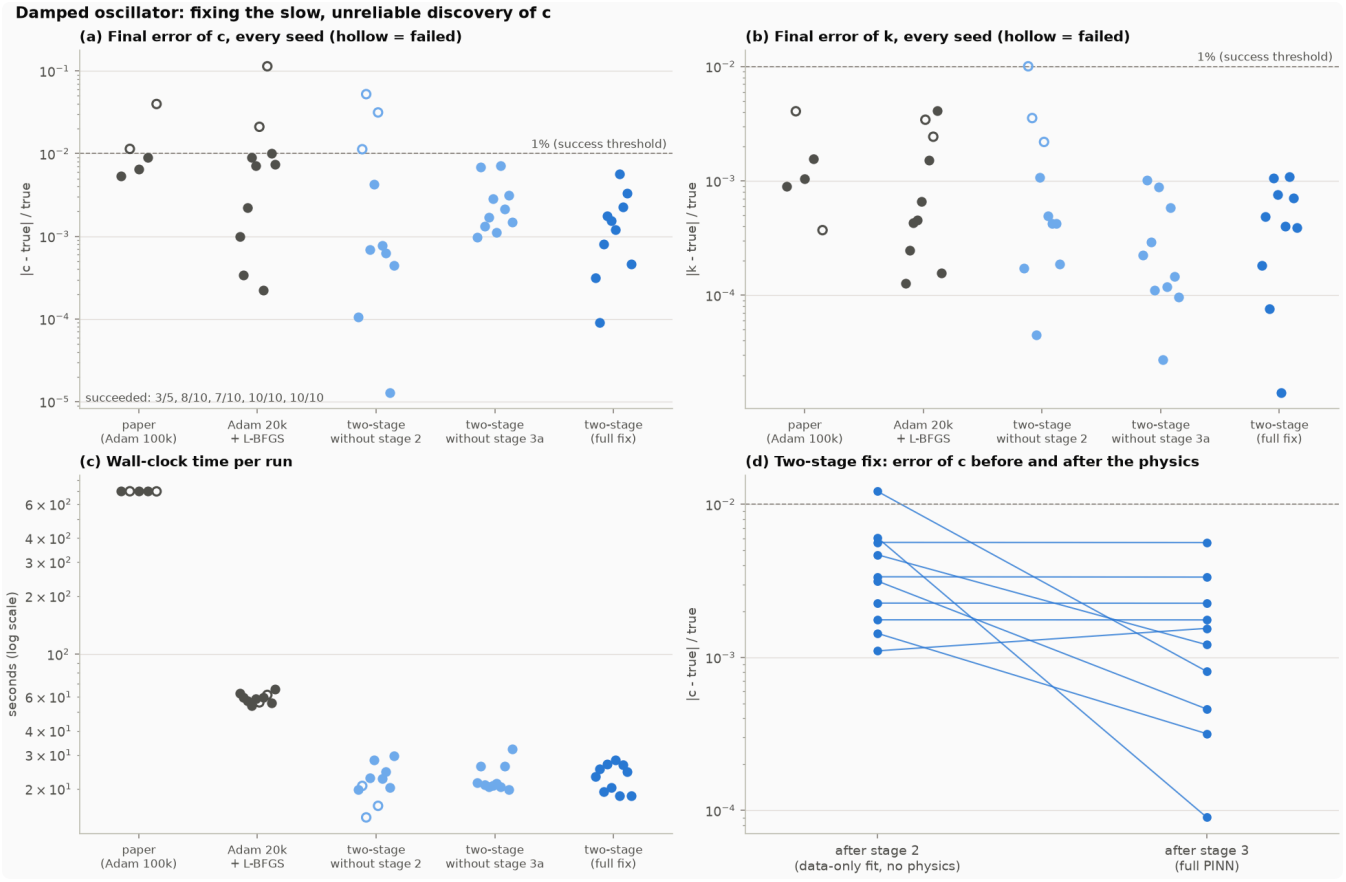


Figure 1. Oscillator results. (a, b) Final error in c and k for every seed; hollow markers are failed runs. (c) Wall-clock time per run. (d) Error in c for the two-stage method after the data-only fit (step 2) and after the full PINN.

The two-stage method succeeds on every seed and is 30 times faster than the paper's setup (699 s to 23 s per run). The ablations show which step matters. Without step 2, the reading of c and k from the data fit, success drops to 7 of 10. Step 3, freezing c and k before releasing them, adds only a small safety margin: without it we got 10 of 10 seeds on one machine and 9 of 10 on another. Step 2 on its own already gives c within 0.33% (median), which is the fit-then-estimate idea used for decades in classical parameter estimation [11]. The PINN stage then refines that estimate, as panel (d) shows.

4.2 Consolidation: the error at the corner

The paper reports a "perfect match" with a maximum error of about 0.045. Over 5 seeds, our median maximum error for $t \geq 0.01$ is 0.061 and the worst is 0.166. The predicted pressure also overshoots to 1.11, although physically it can never exceed 1.

The cause is the corner $x = 0$, $t = 0$. There the pressure must drop from 1 to 0 instantly, so the exact solution has an infinitely steep front at that point. The first 2% of the time window ($t < 0.01$) contains only 11 of the 500 random training points, but 59% of the squared error. On fresh points the equation residual is 49, against 5.5e-6 at the training

points. The training loss therefore looks converged while the solution near the corner is wrong. More training does not help: the paper's loss curve is flat after about 5k iterations, so its last 40k Adam iterations add little.

We compared two fixes. The first is residual-based adaptive sampling (RAD) [4], which repeatedly re-draws training points where the residual is largest. This is the "adaptive re-meshing" that the paper lists as future work. The second is our corner-enriched output:

$$p(x, t) = \mathrm{erf}(x / 2\sqrt{t}) + x \cdot t \cdot N(x, t) \quad (2)$$

The erf term is the exact solution for a soil layer with no base [9]. It carries the jump at $t = 0$ by itself, so the network only has to learn the smooth effect of the sealed base. The factor x keeps $p = 0$ at the top, as in the paper's own output, and the factor t keeps the initial condition exact. Table 4 compares the two fixes with the paper's setup.

Table 4. Consolidation results over 5 seeds. The maximum error is taken over $t \geq 0.01$.

Method	Relative L2 error, median (worst)	Max error	Highest pressure (physical max 1)	Residual on fresh points	Time per run
Paper (Adam 50k)	1.4e-2 (4.3e-2)	6.1e-2	1.108	4.9e+1	1,143 s
Baseline (Adam 10k)	1.7e-2 (3.2e-2)	9.7e-2	1.191	1.9e+2	433 s
Adaptive sampling (RAD)	1.3e-2 (3.8e-2)	3.3e-2	1.076	3.3e+1	395 s
Corner-enriched output	6.4e-5 (9.6e-5)	2.9e-4	1.000	8.4e-7	168 s
Enriched + RAD	5.4e-5 (8.0e-5)	2.1e-4	1.000	3.6e-7	160 s

RAD triples the number of points near the corner (from 11 to 37), but the error barely changes. No placement of points lets a smooth network represent an infinitely steep front. The enriched output removes the need to represent it at all. It lowers the relative error about 200-fold, removes the overshoot and trains 7 times faster than the paper's setup. Adding RAD on top of it gives only a small further gain.

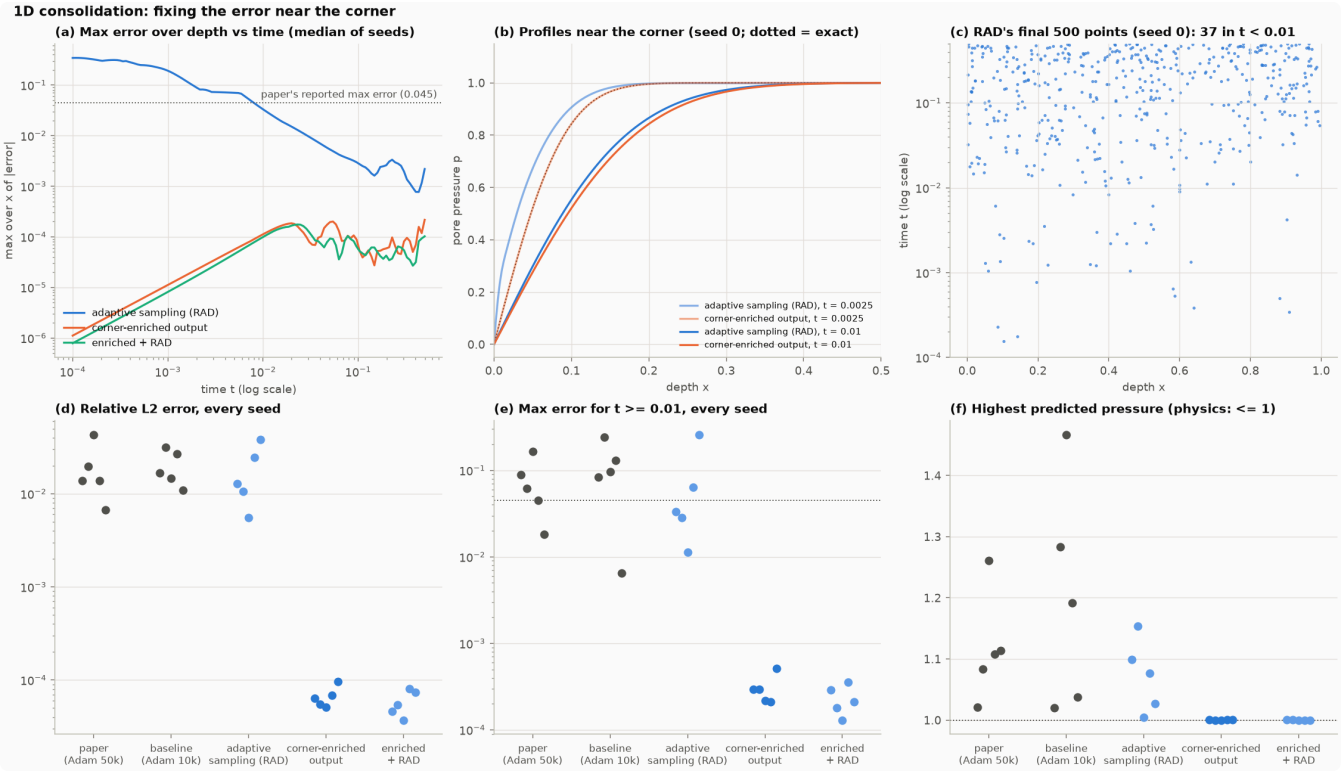


Figure 2. Consolidation results. (a) Maximum error over depth against time. RAD stays above the paper's reported error until t ≈ 0.01, while the enriched output stays at about 2e-4 or less throughout. (b) Pressure profiles near the corner. (c) Where RAD placed its training points. (d–f) Error and highest predicted pressure for every seed.

4.3 Heat conduction: edges, then corners

The paper's network enforces T = 0 on the edges only as a penalty in the loss (a "soft" boundary condition). The paper reports a maximum error of about 5.2e-3, located on the edges. We measured a median of 3.8e-3, also on the edges. 60% of the squared error lies within 0.05 of an edge, a strip that covers only 10% of the plate. The predicted temperature also dips to −3.8e-3, below the physical minimum of 0.

Our first fix is the exact boundary condition that the paper suggests but does not test:

$$T(x, y) = (1 - x^2)(1 - y^2) \cdot N(x, y) \quad (3)$$

The factor in front is zero on all four edges, so T = 0 holds there whatever the network outputs. This lowers the error about 30-fold, and the temperature no longer goes below 0.

With the boundary fixed, a second problem appears. The residual on fresh points is now about 100 times the training loss, and almost all of it sits at the four corners. The cause is a contradiction in the problem itself. Along the top edge T = 0, so $T_{xx} = 0$ there. Along the side edge T = 0 as well, so $T_{yy} = 0$, and the equation then forces $T_{xx} = -1$. At the corner, T_{xx} would have to be both 0 and −1. The exact solution resolves this with a value that depends on the direction of approach: T_{xx} tends to $-2\theta/\pi$, where θ is the angle measured from the horizontal edge. A smooth network has a single value at each point, so

it must break something. The soft-boundary network breaks T = 0, and its largest error sits at a corner in 5 of 5 seeds. The hard-boundary network breaks the equation instead: its residual is exactly 1 at each corner, whatever its weights. Both facts are properties of the problem rather than of training, and our unit tests check them without training a network.

Our second fix adds, at each corner, the exact local solution for a 90° corner. We derived it by hand from the corner geometry alone, not from the series solution:

$$u = r^2 \left[-1/4 + \cos(2\theta)/4 - \left(\ln r \cdot \sin(2\theta) + \theta \cdot \cos(2\theta) \right) / \pi \right] \quad (4)$$

Here r and θ are polar coordinates centred on the corner. The function u is zero on both edges and gives $T_{xx} = -2\theta/\pi$ exactly. Each corner term is faded out away from its corner and still vanishes on every edge, so the boundary condition stays exact and the network only learns a smooth remainder. This is the same idea as the erf term in consolidation. Table 5 summarises the four configurations.

Table 5. Heat conduction results over 5 seeds. BC stands for boundary condition. The last column but one is the ratio of the residual on fresh points to the residual at the training points.

Configuration	Relative L2 error, median (worst)	Max error	Distance from max error to nearest corner	Squared error within 0.15 of a corner (1.8% of area)	Fresh / training residual	Time per run
Paper: soft BC, Adam 50k	1.3e-3 (1.6e-3)	3.8e-3	on the edge	not measured	2.2x	1,983 s
Soft BC, Adam 5k	1.5e-3 (2.5e-3)	4.8e-3	0.00 (at a corner)	26%	1.5x	356 s
Hard BC	4.2e-5 (7.3e-5)	7.7e-5	0.06	18%	97.7x	489 s
Hard BC + corner terms	3.9e-5 (1.1e-4)	2.7e-5	0.87	2%	1.8x	945 s

The corner terms remove the corner hotspot. The maximum error moves away from the corners and falls by a further factor of 3, and the fresh-to-training residual ratio drops from 98 to 1.8, so the training loss can be trusted again. The average error barely changes, and training takes about twice as long. Rerunning the earlier configurations reproduced their errors bit for bit.

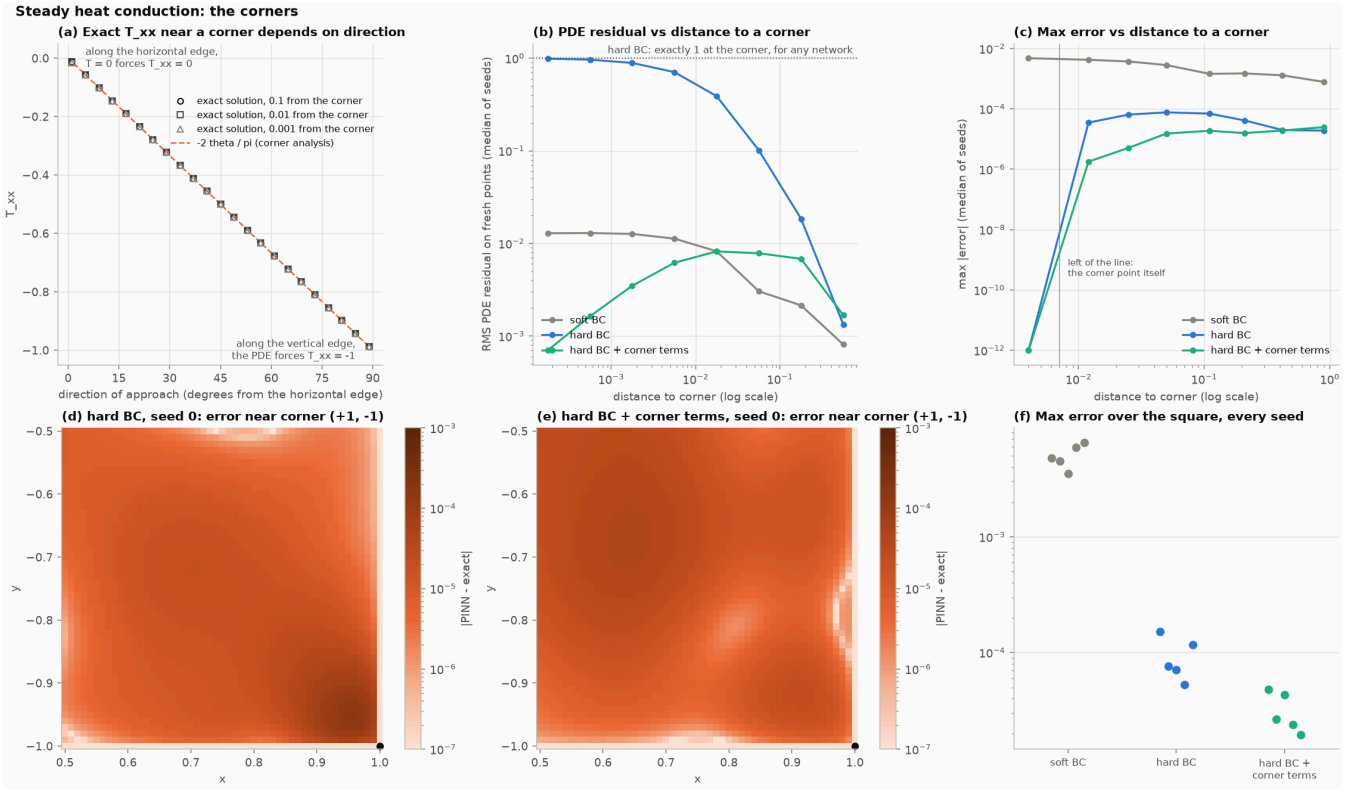


Figure 3. Heat conduction at the corners. (a) The exact T_xx depends on the direction of approach and matches $-2\theta/\pi$. (b) The hard-BC residual rises to exactly 1 at the corner; the corner terms remove the rise. (c) Maximum error against distance to a corner. (d, e) Error near one corner without and with the corner terms. (f) Maximum error for every seed.

4.4 Noisy and sparse data

The paper tests parameter discovery only on clean data with 50 points, and it compares the PINN with no other method. Real measurements are noisy and often sparse, so we tested how far the result holds and how it compares with simpler methods.

We used three data settings: 50 points over [0, 15], as in the paper; 15 points over [0, 15], which is fewer than 3 per oscillation; and 50 points over [0, 5] only. To each we added Gaussian noise of 0%, 1%, 3% or 10% of the initial displacement, with 10 random draws per case. Four methods saw exactly the same data and started from c = 0.2, k = 2:

1. A classical fit, which adjusts c and k until the numerical solution of the ODE matches the data.
2. A plain neural network, which fits the data and then reads c and k off the fitted curve's derivatives.
3. A standard PINN, trained with Adam 20k and then L-BFGS.
4. Our two-stage PINN from Section 4.1.

As a benchmark we computed the Cramer-Rao bound [12], the smallest error that any unbiased method can reach at a given noise level. The bound shows that a fixed success threshold stops being fair as noise grows: with 3% noise, even an ideal method misses c by 1.3% half of the time.

Table 6. Oscillator results with noisy and sparse data. "Runs within 1%" counts clean-data runs over all three data settings. The c errors are relative medians over 10 draws at 1% noise.

Method	Runs within 1%, clean data	c error: 50 points	c error: 15 points	c error: 50 points on [0, 5]	Time per dataset
Ideal (Cramer-Rao bound)	–	4.3e-3	8.1e-3	4.4e-3	–
Classical ODE fit	30/30	5.5e-3	1.2e-2	4.6e-3	0.2 s
Plain neural network	10/30	6.8e-3	4.8e-1	1.6e-2	13 s
Standard PINN	26/30	8.6e-3	1.5e-2	3.3e-2	61 s
Two-stage PINN	23/30	4.6e-3	2.7e-1	6.5e-3	38 s

The classical fit is the best method in every setting. It is also by far the fastest, at 0.2 s per dataset, and stays close to the bound throughout. When the equation is known and has only two unknown parameters, a PINN is not the right tool.

Among the neural methods, the two-stage PINN is the most accurate on dense data: with 50 points and 1% noise it comes within 7% of the bound. It fails on sparse data, however. With only 15 points, the data-only fit in its first step is poor, so the warm start points to the wrong c and k; there the standard PINN is the better choice. At 10% noise every neural method degrades, with c errors 3 to 6 times above the bound, while the classical fit stays within 1.5 times of it.

The physics term does pay off when the data stop early. With clean data only up to t = 5, the plain network's curve over the whole interval [0, 15] is off by 83%, while both PINNs stay within 13 to 22%.

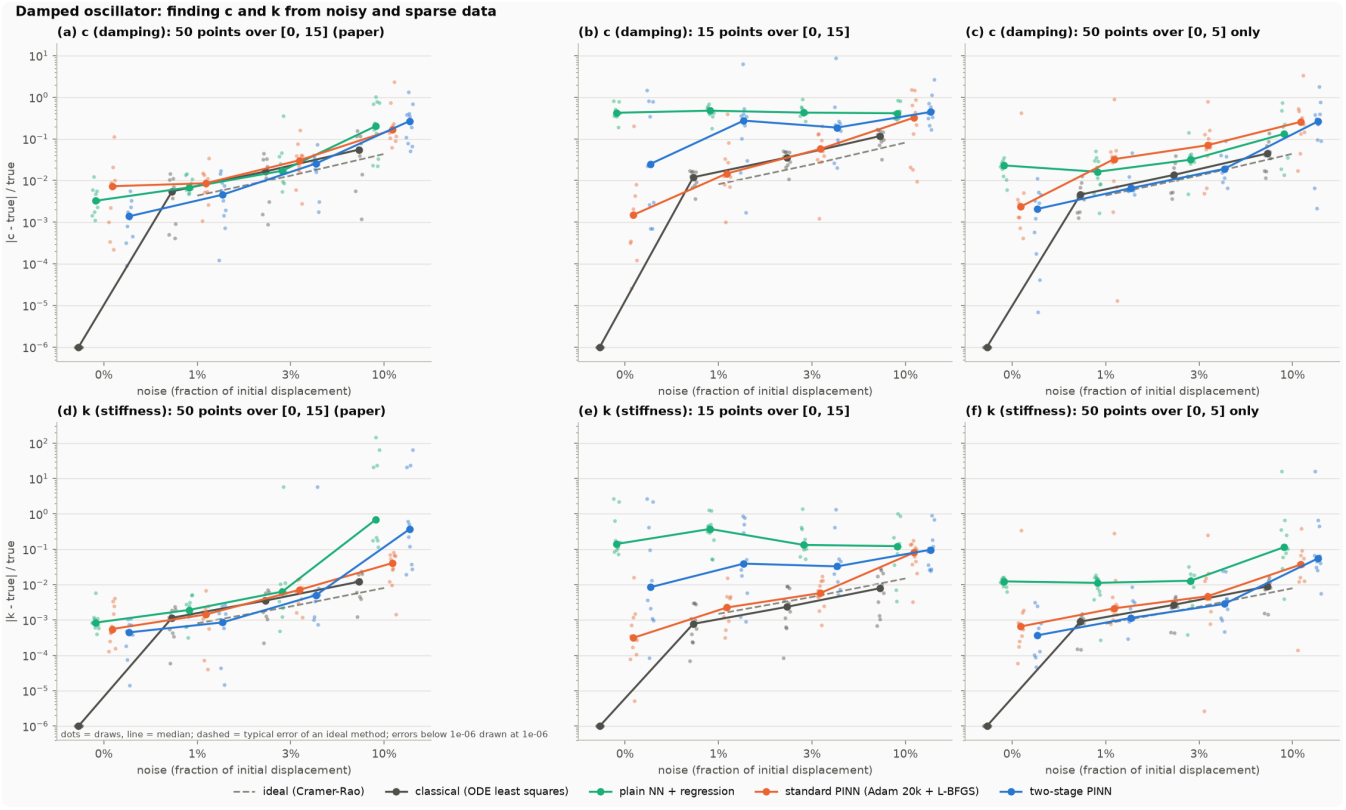


Figure 4. Noisy and sparse data. Relative error in c (top row) and k (bottom row) against noise level, for each data setting. Lines are medians over 10 draws, dots are individual draws, and the dashed line is the Cramer-Rao bound.

5. Discussion

5.1 Comparison with the base paper

Table 7 sets our work against the base paper, area by area.

Table 7. What this project adds to the base paper.

Area	Base paper	This project	Effect
Evidence	One run per experiment	5 to 10 seeds per configuration; median and worst case	Showed that 2 of 5 oscillator runs and most consolidation runs miss the paper's accuracy

Area	Base paper	This project	Effect
Reference solutions	Fourier series (consolidation); finite elements and "no analytical solution" (heat)	Exact solutions for both forward problems, each checked independently; series solution for heat	Errors measured exactly, down to 1e-9
Error analysis	Error maps from one run	Error by region (corner, edge, early time); residual on fresh vs training points	Found where and why each PINN fails
Oscillator	Adam 100k; c found slowly and unreliably	Diagnosis through implied parameters; two-stage warm start	3/5 to 10/10 success; 30x faster
Consolidation	Output multiplied by x	Corner-enriched output $\mathrm{erf}(x / 2\sqrt{t}) + x \cdot t \cdot N$; compared with adaptive sampling	About 200x lower error; no overshoot; 7x faster
Heat: edges	Soft penalty; exact boundary condition suggested but not tested	Output $(1 - x^2)(1 - y^2) \cdot N$	30x lower error; no negative temperatures
Heat: corners	Not identified	Proof that every smooth network fails at the corners; exact corner terms	Corner hotspot removed; max error 3x lower again
Adaptive sampling	Listed as future work	RAD implemented and tested	Little gain alone; small gain on top of enrichment
Robustness	Clean, dense data only	3 data settings x 4 noise levels x 10 draws; classical and plain-network baselines; Cramer-Rao bound	Showed when a PINN is worth using and when a classical fit is better

Area	Base paper	This project	Effect
Engineering	Not described	float64; Adam then L-BFGS with divergence guard; per-term loss logs; 49 unit tests; resumable parallel runs	Bit-identical reruns on the same machine

5.2 What we learned

The main lesson is that a PINN's accuracy is decided by the few places where the physics is hardest: a corner, an edge or a slowly decaying tail. Averages and the training loss both hide these places. In consolidation and heat, the loss looked converged while the equation was badly violated near the corners, and only the residual on fresh points showed it. We now check that residual as a matter of routine.

The second lesson is that known structure belongs in the network's output rather than in the loss. The exact boundary condition, the erf term and the corner terms each did better than adaptive sampling or longer training. Each also trained faster than the paper's setup, except the heat corner terms, which roughly double the time per run. Inverse problems benefit from the same thinking applied to the starting point: reading c and k off a data-only fit turned a method that worked in 3 of 5 seeds into one that works in 10 of 10.

The third lesson is about evidence. The paper's headline results hold for only some seeds, and a method that looks reliable on 5 seeds can fail on 10. Finally, a PINN is not always the right tool. For a known ODE with two parameters, a classical least-squares fit is faster and more accurate. PINNs earn their place when the data are incomplete and the physics has to fill the gaps, as in the extrapolation test of Section 4.4.

5.3 Limitations and future work

The erf and corner terms rely on an analytical understanding of each problem, so they do not carry over automatically to new geometries. A natural next step is to derive such terms automatically from the local form of the equation at a corner or a discontinuity. We have not yet produced uncertainty estimates (error bars) for c and k, and we plan to add them. All timings come from one laptop CPU (Intel i5-13450HX), so they compare methods with each other but do not measure absolute speed. Finally, the study covers only the paper's three problems.

6. Conclusion

We reproduced all three experiments of the base paper over several seeds and against exact solutions. Each problem had one dominant source of error, found in a small region that a smooth network cannot represent. Building the known behaviour of the solution into the network removed that error: the oscillator parameters are now found on every seed and 30 times faster, the consolidation error is about 200 times lower, and the worst-case heat error is about 140 times lower. We also showed where PINNs stop being the best choice: on a known ODE with few parameters, a classical fit does better, especially when the data are noisy.

References

1. Q. Zhang, Y. Chen, Z. Yang. Data Driven Solutions and Discoveries in Mechanics Using Physics Informed Neural Network. Stanford CS229 project report.
2. M. Raissi, P. Perdikaris, G. E. Karniadakis. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 378:686–707, 2019.
3. I. E. Lagaris, A. Likas, D. I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. IEEE Transactions on Neural Networks, 9(5):987–1000, 1998.
4. C. Wu, M. Zhu, Q. Tan, Y. Kartha, L. Lu. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. Computer Methods in Applied Mechanics and Engineering, 403:115671, 2023.
5. D. P. Kingma, J. Ba. Adam: a method for stochastic optimization. ICLR, 2015.
6. D. C. Liu, J. Nocedal. On the limited memory BFGS method for large scale optimization. Mathematical Programming, 45:503–528, 1989.
7. X. Glorot, Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. AISTATS, 2010.
8. K. Terzaghi. Theoretical Soil Mechanics. Wiley, 1943.
9. H. S. Carslaw, J. C. Jaeger. Conduction of Heat in Solids, 2nd edition. Oxford University Press, 1959.
10. S. P. Timoshenko, J. N. Goodier. Theory of Elasticity, 3rd edition. McGraw-Hill, 1970.
11. J. M. Varah. A spline least squares method for numerical parameter estimation in differential equations. SIAM Journal on Scientific and Statistical Computing, 3(1):28–46, 1982.
12. S. M. Kay. Fundamentals of Statistical Signal Processing: Estimation Theory. Prentice Hall, 1993.

Appendix A. Reproducing the results

Every number in this report comes from scripts in our repository, github.com/DBS13062006/pinn-mechanics. The code runs on Python 3.12 and PyTorch, on CPU or CUDA, and the README gives setup steps for a Windows virtual environment, conda and Google Colab. 49 unit tests cover the engine, the exact solutions and the corner facts of Section 4.3, and run in seconds.

Every run seeds all random number generators and saves its library versions, hardware details, loss history, weights and metrics next to its results. Long experiments run as parallel processes and can resume after an interruption without redoing finished runs. Table A1 lists the cost of each experiment. The networks are small, so we ran them as parallel CPU jobs on an Intel i5-13450HX (10 cores) with 16 GB of RAM; the machine also has an NVIDIA RTX 4050.

Table A1. Runs and wall-clock time for each experiment.

Experiment	Runs	Parallel jobs	Wall time
Oscillator: reproduction	15	5	15 min
Oscillator: two-stage fix and ablations	40	5	5 min
Oscillator: noisy and sparse data	360	4	50 min
Consolidation: reproduction	10	5	29 min
Consolidation: enrichment vs adaptive sampling	15	5	13 min
Heat: reproduction and exact boundary condition	15	3	82 min
Heat: corners	15	3	50 min